

1. What is Angular?

Angular is a **front-end framework** developed by Google used to build **dynamic, scalable Single Page Applications (SPA)** using **TypeScript**.

👉 It follows:

- Component-based architecture
- MVC-like pattern
- Dependency Injection

Example

A typical Angular app:

- Login page
- Dashboard
- API integration with backend (.NET)

Instead of reloading pages → Angular updates UI dynamically.

2. Difference between AngularJS and Angular

Feature	AngularJS	Angular
Language	JavaScript	TypeScript
Architecture	MVC	Component-based
Performance	Slower	Faster
Mobile Support	Poor	Strong
Data Binding	Two-way	One-way (default)
Dependency Injection	Basic	Advanced
Version	1.x	2+ (modern Angular)

👉 Example:

- AngularJS: `$scope.name`
- Angular: `{{ name }}`

3. Key Features of Angular

◇ 1. Component-Based Architecture

UI is broken into reusable components.

◇ 2. Two-Way Data Binding

Keeps UI and data in sync.

◇ 3. Dependency Injection

Manages services efficiently.

◇ 4. Routing

Supports navigation without reload.

◇ 5. RxJS Integration

Handles async operations.

◇ 6. TypeScript Support

Strong typing → fewer bugs

Example

```
@Component({
  selector: 'app-user',
  template: `<h1>{{name}}</h1>`
})
export class UserComponent {
  name = 'Vignesh';
}
```

4. What is SPA (Single Page Application)?

A **Single Page Application (SPA)** loads **only one HTML page** and dynamically updates content without refreshing the page.

Example:

Apps like:

- Gmail
- Netflix
- Dashboard apps

👉 Angular handles routing internally → no full page reload.

Flow:

1. Load app once
2. Navigate via routes
3. Only content updates

5. What is Angular CLI?

Angular CLI (Command Line Interface) is a tool to:

- Create projects
- Generate components/services
- Build & run apps

👉 Developed by Google

Example Commands

ng new my-app

ng serve

ng generate component user

ng build

👉 Real Use:

Saves time → no manual setup

🔗 6. What is a Component?

A **component** is the **building block of Angular UI**.

It controls:

- HTML (template)
- CSS (styles)
- Logic (TypeScript)

Example

```
@Component({
  selector: 'app-hello',
  template: `<h1>Hello {{name}}</h1>`
})
export class HelloComponent {
  name = 'Angular';
}
```

👉 This component displays:

👉 Hello Angular

📦 7. What is a Module?

A **module (NgModule)** is a **container** that groups related components, directives, pipes, and services.

Example

```
@NgModule({  
  declarations: [AppComponent],  
  imports: [BrowserModule],  
  bootstrap: [AppComponent]  
})  
export class AppModule {}
```

👉 Purpose:

- Organize application
- Manage dependencies

🏠 8. What is AppModule?

AppModule is the **root module** of the Angular application.

👉 It is the **starting point (entry module)**

Example

```
@NgModule({  
  declarations: [AppComponent],  
  imports: [BrowserModule],  
  bootstrap: [AppComponent]  
})  
export class AppModule {}
```

👉 bootstrap → tells Angular which component to load first

🔗 9. What are Decorators in Angular?

Decorators are **special functions** that add metadata to classes.

They tell Angular **how a class should behave**.

Common Decorators

- @Component
- @NgModule
- @Injectable
- @Input
- @Output

Example

```
@Component({  
  selector: 'app-user',  
  template: '<h1>User</h1>'  
})  
export class UserComponent {}
```

👉 @Component → tells Angular this is a component

🔑 10. What is Metadata in Angular?

Metadata is the **information passed inside decorators**.

It defines:

- Selector
- Template
- Styles
- Dependencies

Example

```
@Component({  
  selector: 'app-user',  
  templateUrl: './user.component.html',
```

```
styleUrls: ['./user.component.css']  
})
```

👉 Here:

- selector
- templateUrl
- styleUrls

👉 All these = **metadata**

11. What is Data Binding?

Data binding is the mechanism that connects:

👉 **Component (TypeScript logic) ↔ Template (HTML UI)**

It allows data to flow between the UI and the component.

Example

```
export class AppComponent {  
  name = 'Vignesh';  
}  
<h1>{{ name }}</h1>
```

👉 Output: Vignesh

Interview Tip:

Data binding keeps UI and data synchronized automatically.

📌 12. Types of Data Binding in Angular

There are **4 types**:

1. Interpolation (Component → View)

```
<h1>{{ name }}</h1>
```

2. Property Binding (Component → View)

```
<img [src]="imageUrl">
```

3. Event Binding (View → Component)

```
<button (click)="handleClick()">Click</button>
```

4. Two-Way Binding (Both directions)

```
<input [(ngModel)]="name">
```

Summary

Type	Direction
Interpolation	Component → View
Property Binding	Component → View
Event Binding	View → Component
Two-way Binding	Both

13. Difference between Interpolation and Property Binding

Feature	Interpolation	Property Binding
Syntax	{{ value }}	[property]="value"
Use	Display text	Set DOM properties
Type	String only	Any type

Example `<h1>{{name}}</h1>` `<img [src]="url">`

Example

```
<h1>{{ name }}</h1>      <!-- Interpolation -->
<img [src]="imageUrl">    <!-- Property Binding -->
```

Interview Tip:

Use interpolation for text, property binding for DOM properties.

⚡ 14. What is Event Binding?

Event binding allows the **UI to send data to the component**.

Example

```
<button (click)="onClick()">Click Me</button>
onClick() {
  console.log('Button clicked');
}
```

Common Events

- click
- input
- change
- submit

Interview Tip:

Event binding uses () syntax.

15. What is Two-Way Data Binding?

Two-way binding means:

- 👉 UI changes → Component updates
- 👉 Component changes → UI updates

Example

```
<input [(ngModel)]="name">  
<p>{{ name }}</p>
```

- 👉 If user types → value updates automatically

Behind the scenes:

`[(ngModel)] = [value] + (input event)`

Interview Tip:

Requires FormsModule

16. What is ngModel?

ngModel is a directive used for **two-way data binding**.

Example

```
<input [(ngModel)]="username">  
username = "";
```

Requirement

```
import { FormsModule } from '@angular/forms';
```

Interview Tip:

ngModel is mainly used in **template-driven forms**

17. What is Template Reference Variable?

A **template reference variable** is used to access a DOM element directly in the template.

Example

```
<input #nameInput>
<button (click)="show(nameInput.value)">Submit</button>
show(value: string) {
  console.log(value);
}
```

Use Cases

- Get input value
- Access DOM element

Interview Tip:

Defined using #variableName

18. What is Safe Navigation Operator (?.)?

It prevents **null or undefined errors** in templates.

Problem

```
<p>{{ user.name }}</p>
```

✗ Error if user = null

Solution

```
<p>{{ user?.name }}</p>
```

✓ No error

Interview Tip:

Also called **Elvis operator**

🔗 19. What is \$event in Angular?

\$event is a **special variable** that contains event data.

Example

```
<input (input)="onInput($event)">
onInput(event: any) {
  console.log(event.target.value);
}
```

Shortcut

```
<input (input)="onInput($event.target.value)">
```

Interview Tip:

\$event gives access to DOM event object

🔍 20. Difference between Attribute Binding vs Property Binding

Feature	Attribute Binding	Property Binding
Syntax	[attr.name]	[property]
Works on	HTML attributes	DOM properties
Use case	ARIA, colspan	value, src, disabled
Example	[attr.colspan]="2"	[value]="name"

Example

<td [attr.colspan]="2"></td>	<!-- Attribute -->
<input [value]="name">	<!-- Property -->

Key Difference

- Attribute → HTML level
- Property → DOM level (dynamic)

Interview Tip:

Always prefer property binding unless attribute is required.

🔗 21. What is a Directive?

A **directive** is a class in Angular that **modifies the behavior or appearance of DOM elements**.

👉 In simple terms:

Directives control how elements behave in the UI.

Example

```
<p *ngIf="isVisible">Hello</p>
```

👉 *ngIf is a directive that controls whether the element is shown.

Interview Tip:

Components are also directives with templates.

📖 22. Types of Directives

Angular has **3 types**:

1. Component Directive

- Has template
- Example: @Component

2. Structural Directives

- Change DOM structure (add/remove elements)
- Examples:
 - *ngIf
 - *ngFor
 - *ngSwitch

3. Attribute Directives

- Change appearance or behavior
- Examples:
 - ngClass
 - ngStyle

23. Structural vs Attribute Directives

Feature	Structural Directive	Attribute Directive
Purpose	Change DOM structure	Change style/behavior
Adds/removes elements	✅ Yes	❌ No
Syntax	* prefix	No *
Example	*ngIf, *ngFor	ngClass, ngStyle

Example

```
<div *ngIf="isLoggedIn">Welcome</div> <!-- Structural -->
<div [ngClass]="{active: true}"></div> <!-- Attribute -->
```

Interview Tip:

* means Angular rewrites DOM internally.

24. How does *ngIf work internally?

When you write:

```
<div *ngIf="isVisible">Hello</div>
```

Angular converts it internally into:

```
<ng-template [ngIf]="isVisible">
  <div>Hello</div>
</ng-template>
```

How it works:

- If isVisible = true → element added to DOM
- If false → element completely removed

Interview Tip:

*ngIf does NOT hide elements—it removes them from DOM.

🔍 25. Difference between *ngIf and [hidden]

Feature	*ngIf	[hidden]
Removes element	✅ Yes	❌ No
Just hides	❌ No	✅ Yes
Performance	Better (less DOM)	More DOM
Use case	Conditional rendering	Temporary visibility

Example

```
<div *ngIf="false">Hidden</div> <!-- Not in DOM -->
<div [hidden]="true">Hidden</div> <!-- Still in DOM -->
```

Interview Tip:

Use *ngIf for performance-critical scenarios.

📦 26. What is *ngFor?

*ngFor is used to **loop through collections (arrays)** and render elements.

Example

```
users = ['A', 'B', 'C'];
```

```
<ul>
```

```
  <li *ngFor="let user of users">{{ user }}</li>
```

```
</ul>
```

Output

A

B

C

Advanced Example

```
<li *ngFor="let user of users; let i = index">
```

```
  {{ i }} - {{ user }}
```

```
</li>
```

27. What is trackBy?

trackBy improves performance by helping Angular **identify items uniquely**.

Problem (without trackBy)

- Angular re-renders entire list

Solution

```
<li *ngFor="let user of users; trackBy: trackById">
```

```
  {{ user.name }}
```

```
</li>
```

```
trackById(index: number, user: any) {
```

```
  return user.id;
```

```
}
```

Interview Tip:

trackBy prevents unnecessary DOM re-rendering.

28. What is ngSwitch?

ngSwitch is used for **conditional rendering (multiple conditions)**.

Example

```
<div [ngSwitch]="role">
  <p *ngSwitchCase="admin">Admin</p>
  <p *ngSwitchCase="user">User</p>
  <p *ngSwitchDefault>Guest</p>
</div>
```

Use Case:

- Role-based UI
- Status-based rendering

Interview Tip:

Cleaner than multiple *ngIf

29. How to create a custom directive?

You can create your own directive to modify DOM behavior.

Example: Highlight Directive

ng generate directive highlight

Code

```
import { Directive, ElementRef } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  constructor(el: ElementRef) {
    el.nativeElement.style.backgroundColor = 'yellow';
  }
}
```

Usage

<p appHighlight>Highlighted Text</p>

Interview Tip:

Custom directives are used for reusable behavior.

🔗 30. What is HostBinding and HostListener?

These are used inside directives to interact with DOM.

◇ HostBinding

Binds a property of the host element.

Example

```
import { Directive, HostBinding } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {

  @HostBinding('style.color') color = 'red';

}
```

👉 Automatically sets text color

◆ HostListener

Listens to DOM events

Example

```
import { Directive, HostListener } from '@angular/core';

@Directive({
  selector: '[appHover]'
})
export class HoverDirective {

  @HostListener('mouseenter') onMouseEnter() {
    console.log('Mouse entered');
  }

  @HostListener('mouseleave') onMouseLeave() {
    console.log('Mouse left');
  }

}
```

Interview Tip:

- HostBinding → binds properties
- HostListener → listens to events

Final Summary (Interview Ready)

- Directive = modifies DOM behavior
- Types:
 - Structural → changes DOM (*ngIf)
 - Attribute → changes style (ngClass)
- *ngIf removes element
- *ngFor loops data
- trackBy improves performance
- Custom directives → reusable logic
- HostBinding & HostListener → DOM interaction

31. What is Component Lifecycle?

The **component lifecycle** is the sequence of stages a component goes through:

 Creation → Rendering → Updating → Destruction

Angular provides **lifecycle hooks** to tap into these stages.

Flow (important to say in interview)

1. Component created
2. Inputs initialized
3. View rendered
4. Change detection runs
5. Component destroyed

32. List all lifecycle hooks in Angular

Order matters (interview question ):

1. constructor
2. ngOnChanges
3. ngOnInit
4. ngDoCheck
5. ngAfterContentInit
6. ngAfterContentChecked
7. ngAfterViewInit
8. ngAfterViewChecked
9. ngOnDestroy

Visual Order

constructor → ngOnChanges → ngOnInit → ngDoCheck
→ ngAfterContentInit → ngAfterContentChecked
→ ngAfterViewInit → ngAfterViewChecked → ngOnDestroy

⚡ 33. When does ngOnInit get called?

ngOnInit() is called:

- 👉 After component initialization
- 👉 After first ngOnChanges

Example

```
export class AppComponent implements OnInit {  
  
  ngOnInit() {  
    console.log('Component initialized');  
  }  
  
}
```

Use Case

- API calls

- Initial data loading

Interview Tip:

ngOnInit runs **once only**

34. Difference between ngOnInit and constructor

Feature	Constructor	ngOnInit
Purpose	Object creation	Initialization logic
Angular aware	 No	 Yes
Runs	First	After ngOnChanges
Use	DI setup	API calls

Example

```
constructor(private service: UserService) {  
  console.log('Constructor');  
}
```

```
ngOnInit() {  
  console.log('ngOnInit');  
}
```

Output

Constructor
ngOnInit

Interview Tip:

Never call APIs in constructor.

35. What is ngOnChanges?

Triggered when:

👉 **Input properties change**

Example

```
@Input() name: string;
```

```
ngOnChanges(changes: SimpleChanges) {  
  console.log(changes);  
}
```

Output

```
{  
  name: {  
    previousValue: 'A',  
    currentValue: 'B'  
  }  
}
```

Use Case

- React to input updates

36. What is ngDoCheck?

ngDoCheck() is called:

👉 **During every change detection cycle**

Example

```
ngDoCheck() {  
  console.log('Change detection running');  
}
```

Use Case

- Custom change detection logic

Interview Warning ⚠

Runs frequently → can affect performance

🔪 37. When to use ngOnDestroy?

Called when:

- 👉 Component is destroyed (removed from DOM)

Example

subscription: any;

```
ngOnInit() {  
  this.subscription = this.service.getData().subscribe();  
}
```

```
ngOnDestroy() {  
  this.subscription.unsubscribe();  
}
```

Use Cases

- Unsubscribe Observables

- Remove event listeners
- Clear timers

Interview Tip:

Prevents memory leaks

38. What is Change Detection?

Change detection is the process where Angular:

👉 **Checks for data changes and updates the UI**

Example

```
name = 'Vignesh';
```

```
changeName() {  
  this.name = 'Angular';  
}
```

```
<h1>{{ name }}</h1>
```

```
<button (click)="changeName()">Change</button>
```

👉 UI updates automatically

How it works:

- Triggered by:
 - Events
 - HTTP calls
 - Timers

Interview Tip:

Angular uses **zone.js** to detect async changes

39. What is OnPush Strategy?

OnPush is a **change detection optimization technique**

Default Behavior

- Angular checks everything

OnPush Behavior

- Checks only when:
 - Input reference changes
 - Event occurs
 - Observable emits

Example

```
@Component({
  selector: 'app-user',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `{{ user.name }}`
})
export class UserComponent {
  @Input() user: any;
}
```

Benefit

- Better performance

Interview Tip:

OnPush works with **immutable data**

⚙️ 40. How Angular updates the DOM?

Angular updates DOM using:

👉 **Change Detection + Virtual DOM-like mechanism**

Process:

1. Data changes in component
2. Angular detects change
3. Compares previous vs current value
4. Updates only changed parts

Example

```
name = 'A';
```

```
update() {  
  this.name = 'B';  
}  
<h1>{{ name }}</h1>
```

👉 Only text updates—not full page reload

Behind the scenes:

- Uses **zone.js**
- Uses efficient DOM diffing

Interview Tip:

Angular updates only affected DOM nodes (not full refresh)

Final Summary

- Lifecycle = component life stages
- `ngOnInit` → initialization
- `ngOnChanges` → input changes
- `ngDoCheck` → custom detection
- `ngOnDestroy` → cleanup
- Change Detection → updates UI
- `OnPush` → performance optimization

1. Explain Lifecycle Hooks in Order

Interview Answer (Structured)

Angular lifecycle hooks define the **different stages of a component's life**, from creation to destruction.

Order (VERY IMPORTANT)

constructor

→ `ngOnChanges`

→ `ngOnInit`

→ `ngDoCheck`

→ `ngAfterContentInit`

→ `ngAfterContentChecked`

→ `ngAfterViewInit`

→ `ngAfterViewChecked`

→ `ngOnDestroy`

Explanation

- **constructor** → object creation (DI happens here)
- **ngOnChanges** → called when `@Input` changes
- **ngOnInit** → component initialized (runs once)
- **ngDoCheck** → custom change detection
- **ngAfterContentInit** → projected content initialized
- **ngAfterViewInit** → view initialized
- **ngOnDestroy** → cleanup before destruction

Example

export class DemoComponent implements OnInit, OnDestroy {

```
  constructor() {  
    console.log('Constructor');  
  }
```

```
  ngOnInit() {  
    console.log('OnInit');  
  }
```

```
  ngOnDestroy() {  
    console.log('Destroyed');  
  }  
}
```

Pro Tip:

Most used hooks: **ngOnInit**, **ngOnChanges**, **ngOnDestroy**

💧 2. Why use ngOnDestroy?

☑ Interview Answer

ngOnDestroy() is used to **clean up resources** when a component is removed from the DOM.

👉 This prevents:

- Memory leaks
- Unnecessary API calls
- Performance issues

🔍 Real Problem

If you don't unsubscribe → app keeps running background tasks

☑ Example

subscription: any;

```
ngOnInit() {  
  this.subscription = this.http.get('/api/data')  
    .subscribe(res => console.log(res));  
}
```

```
ngOnDestroy() {  
  this.subscription.unsubscribe();  
}
```

🔄 Other Use Cases

- Clear timers (setInterval)
- Remove event listeners

Interview Tip:

Always clean Observables in ngOnDestroy

3. Difference between ngOnInit and constructor

Interview Answer

Feature	constructor	ngOnInit
Purpose	Object creation	Initialization logic
Angular aware	 No	 Yes
Runs	First	After ngOnChanges
Use	Dependency injection	API calls, setup

Example

```
constructor(private service: UserService) {  
  console.log('Constructor');  
}
```

```
ngOnInit() {  
  console.log('ngOnInit');  
}
```

Output

```
Constructor  
ngOnInit
```

Key Point to Say

Constructor is for **injecting dependencies**, ngOnInit is for **business logic**

4. How Change Detection Works?

Interview Answer

Change detection is the process by which Angular:

 **detects data changes and updates the UI automatically**

Flow

1. Data changes in component
2. Angular detects change
3. Compares old vs new value
4. Updates DOM

Example

```
name = 'Vignesh';
```

```
changeName() {  
  this.name = 'Angular';  
}
```

```
<h1>{{ name }}</h1>
```

```
<button (click)="changeName()">Change</button>
```

 Clicking button updates UI automatically

Triggered By

- User events (click)
- HTTP calls

- Timers (setTimeout)
- Observables

Behind the scenes

- Uses **zone.js**
- Runs change detection automatically after async tasks

Interview Tip:

Angular checks component tree and updates only changed values

5. What is OnPush?

Interview Answer

OnPush is a **change detection strategy** used to improve performance.

Default Strategy

- Angular checks **all components**

OnPush Strategy

Angular checks component ONLY when:

1. Input reference changes
2. Event occurs inside component
3. Observable emits new value

Example

```
import { ChangeDetectionStrategy, Component, Input } from '@angular/core';
```

```
@Component({  
  selector: 'app-user',  
  template: `{{ user.name }}`,  
  changeDetection: ChangeDetectionStrategy.OnPush  
})  
export class UserComponent {  
  @Input() user: any;  
}
```

Important

this.user.name = 'New'; //  won't trigger update
this.user = { ...this.user, name: 'New' }; //  works

Key Concept

OnPush works with **immutable data (new reference)**

Benefits

- Better performance
- Less unnecessary checks

Final Interview Summary (Say This Confidently)

- Lifecycle hooks define component stages
- ngOnInit → initialization
- ngOnDestroy → cleanup
- Constructor vs ngOnInit → DI vs logic
- Change detection → auto UI update

- OnPush → performance optimization

41. What is @Input()?

@Input() is used to **receive data from parent to child component**.

Example

Parent Component

```
<app-child [name]="parentName"></app-child>  
parentName = 'Vignesh';
```

Child Component

```
import { Input } from '@angular/core';  
  
export class ChildComponent {  
  @Input() name!: string;  
}  
  
<p>{{ name }}</p>
```

Interview Tip:

@Input() = Parent → Child communication

42. What is @Output()?

@Output() is used to **send data from child to parent**.

☑ Example

Child Component

```
import { Output, EventEmitter } from '@angular/core';

@Output() notify = new EventEmitter<string>();

sendData() {
  this.notify.emit('Hello Parent');
}

<button (click)="sendData()">Send</button>
```

Parent Component

```
<app-child (notify)="receiveData($event)"></app-child>

receiveData(data: string) {
  console.log(data);
}
```

🎯 Interview Tip:

@Output() uses EventEmitter internally

⚡ 43. What is EventEmitter?

EventEmitter is a class used to **emit events from child to parent**.

Example

```
@Output() message = new EventEmitter<string>();

this.message.emit('Hello');
```

🎯 Key Point:

It works like a custom event system

▼ 44. How to pass data Parent → Child?

Use `@Input()`

Example

```
<app-user [user]="userData"></app-user>  
userData = { name: 'Vignesh' };
```

Child:

```
@Input() user: any;
```

🎯 Flow

Parent → binds value → Child receives

▲ 45. How to pass data Child → Parent?

Use `@Output()` + `EventEmitter`

Example

Child:

```
@Output() send = new EventEmitter<string>();
```

```
sendData() {
```

```
this.send.emit('Hello');  
}
```

Parent:

```
<app-child (send)="handle($event)"></app-child>
```

Flow

Child emits → Parent listens

46. How to share data between unrelated components?

Use a **shared service with RxJS (BehaviorSubject)**

Example

Service

```
import { Injectable } from '@angular/core';  
import { BehaviorSubject } from 'rxjs';  
  
@Injectable({ providedIn: 'root' })  
export class DataService {  
  
  private data = new BehaviorSubject<string>('Initial');  
  currentData$ = this.data.asObservable();  
  
  updateData(value: string) {  
    this.data.next(value);  
  }  
}
```

Component A (Sender)

```
constructor(private service: DataService) {}
```

```
send() {  
  this.service.updateData('Hello from A');  
}
```

Component B (Receiver)

```
constructor(private service: DataService) {}
```

```
ngOnInit() {  
  this.service.currentData$.subscribe(data => {  
    console.log(data);  
  });  
}
```

🎯 Interview Tip:

Use **BehaviorSubject** for shared state

🔍 47. What is ViewChild?

ViewChild is used to **access child component or DOM element inside the same template.**

Example

```
<input #inputBox>  
<button (click)="focus()">Focus</button>  
import { ViewChild, ElementRef } from '@angular/core';  
  
@ViewChild('inputBox') input!: ElementRef;  
  
focus() {
```

```
this.input.nativeElement.focus();  
}
```

Use Cases

- Access DOM
- Call child component methods

48. What is ContentChild?

Used to access **content projected using ng-content**

Example

Parent:

```
<app-child>  
  <p #contentRef>Hello</p>  
</app-child>
```

Child:

```
import { ContentChild, ElementRef } from '@angular/core';
```

```
@ContentChild('contentRef') content!: ElementRef;
```

Use Case

- Access projected content

49. Difference between ViewChild and ContentChild

Feature	ViewChild	ContentChild
Access	Template elements	Projected content

Scope	Inside component	From parent
Use	DOM/child component	ng-content

🎯 Simple Way to Say

ViewChild → own template

ContentChild → projected content

📁 50. What is ng-content?

ng-content is used for **content projection** (like slots in other frameworks)

Example

Child Component

```
<div>
  <h1>Header</h1>
  <ng-content></ng-content>
</div>
```

Parent Component

```
<app-child>
  <p>This is projected content</p>
</app-child>
```

Output

Header

This is projected content

Use Case

- Reusable UI components (cards, modals)

Final Summary

- @Input() → Parent → Child
- @Output() → Child → Parent
- EventEmitter → sends events
- Service + BehaviorSubject → shared data
- ViewChild → access DOM
- ContentChild → projected content
- ng-content → content projection

Scenario 1

 "How to share data between unrelated components?"

Interview Answer (Structured)

When components are **not in parent-child relationship**, we use:

Shared Service + RxJS BehaviorSubject

This allows **centralized state management** and **real-time data updates**.

Why BehaviorSubject?

- Holds **latest value**
- New subscribers get **current data immediately**
- Works like a **data stream**

☑ Full Example

◇ Step 1: Create Service

```
import { Injectable } from '@angular/core';
import { BehaviorSubject } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class DataService {

  private messageSource = new BehaviorSubject<string>('Initial Data');
  currentMessage$ = this.messageSource.asObservable();

  updateMessage(message: string) {
    this.messageSource.next(message);
  }
}
```

◇ Step 2: Sender Component

```
constructor(private dataService: DataService) {}

sendData() {
  this.dataService.updateMessage('Hello from Component A');
}
```

◇ Step 3: Receiver Component

```
constructor(private dataService: DataService) {}

ngOnInit() {
  this.dataService.currentMessage$.subscribe(data => {
    console.log('Received:', data);
  });
}
```

Flow

Component A → Service → BehaviorSubject → Component B

Interview Tip

Always say: "We use BehaviorSubject to maintain shared state and emit latest value to all subscribers."

Common Mistake

- Using normal variable instead of observable ❌
- Not unsubscribing (if needed) ❌

Scenario 2

 "How child communicates with parent?"

Interview Answer

Use:

@Output() + EventEmitter

Child emits event → Parent listens

Full Example

◇ Child Component

```
import { Component, Output, EventEmitter } from '@angular/core';
```

```
@Component({  
  selector: 'app-child',
```

```
template: `<button (click)="sendData()">Send</button>`
})
export class ChildComponent {

  @Output() messageEvent = new EventEmitter<string>();

  sendData() {
    this.messageEvent.emit('Hello Parent');
  }
}
```

◆ Parent Component

```
<app-child (messageEvent)="receiveData($event)"></app-child>
receiveData(data: string) {
  console.log('Received from child:', data);
}
```

🔄 Flow

Child → emit() → Parent method

🎯 Interview Tip

@Output creates a custom event that parent can subscribe to.

⚠ Common Mistake

- Forgetting \$event ❌
- Using @Input instead ❌

Scenario 3

"How to access DOM element?"

Interview Answer

Use:

@ViewChild()

It allows accessing:

- DOM elements
- Child components

Example (Access Input Field)

Template

```
<input #inputBox type="text">
<button (click)="focusInput()">Focus</button>
```

Component

```
import { Component, ViewChild, ElementRef, AfterViewInit } from '@angular/core';
```

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
```

```
export class AppComponent implements AfterViewInit {
```

```
  @ViewChild('inputBox') input!: ElementRef;
```

```
  ngAfterViewInit() {
    console.log('View initialized');
  }
}
```

```
  focusInput() {
```

```
this.input.nativeElement.focus();  
}  
}
```

Flow

Template reference (#inputBox) → ViewChild → Component access

Interview Tip

ViewChild is available after **ngAfterViewInit**

Common Mistake

- Accessing ViewChild in ngOnInit ❌

Bonus: Real Interview Follow-up Questions

 "Why BehaviorSubject instead of Subject?"

✓ BehaviorSubject stores latest value

 "Can we use ViewChild to call child methods?"

✓ Yes

```
@ViewChild(ChildComponent) child!: ChildComponent;
```

```
this.child.someMethod();
```

👉 "How to avoid memory leaks in Scenario 1?"

✓ Use:

- async pipe OR
- unsubscribe in ngOnDestroy

🚀 Final Summary (Say This in Interview)

- Unrelated components → Service + BehaviorSubject
- Child → Parent → @Output + EventEmitter
- DOM access → ViewChild

51. What is a Service?

A **service** is a class used to **share business logic, data, or functionality across components**.

👉 Keeps components clean and reusable.

☑ Example

```
import { Injectable } from '@angular/core';
```

```
@Injectable({  
  providedIn: 'root'  
})
```

```
export class UserService {
```

```
  getUsers() {  
    return ['A', 'B', 'C'];  
  }  
}
```

Usage in Component

```
constructor(private userService: UserService) {}
```

```
ngOnInit() {  
  console.log(this.userService.getUsers());  
}
```

Interview Tip

Services handle business logic, components handle UI.

52. What is Dependency Injection (DI)?

Dependency Injection is a design pattern where:

👉 A class gets its dependencies from **outside**, not by creating them.

Without DI ✖

```
const service = new UserService();
```

With DI

```
constructor(private userService: UserService) {}
```

Interview Tip

Angular automatically injects dependencies using its DI system.

⚙️ 53. What is Injector?

An **injector** is a system that:

👉 **Creates and provides instances of services**

How it works

Component → asks for service → Injector provides instance

Example

```
constructor(private userService: UserService) {}
```

👉 Injector gives UserService instance

🎯 Interview Tip

Injector is responsible for managing service instances.

🌳 54. What is Hierarchical DI?

Angular DI is **hierarchical**, meaning:

👉 There are multiple injectors at different levels

Levels

1. Root Injector (App level)
2. Module Injector
3. Component Injector

Example

```
@Component({  
  providers: [UserService]  
})
```

👉 Creates **new instance for that component only**

🎯 Key Point

Root → Shared instance

Component → New instance

Interview Tip

Hierarchical DI controls scope of services

🌐 55. What is providedIn: 'root'?

It means:

👉 Service is available **application-wide (singleton)**

Example

```
@Injectable({  
  providedIn: 'root'  
})  
export class UserService {}
```

Benefits

- No need to add in providers array
- Tree-shakable (optimized)

Interview Tip

This is the recommended way to register services.

56. Difference between Service and Component

Feature	Service	Component
Purpose	Business logic	UI
Template	✗ No	✓ Yes
Reusability	High	Limited
Decorator	@Injectable	@Component

Example

- Service → API call
- Component → Display UI

Interview Tip

Separation of concerns

57. What are Providers?

Providers tell Angular:

 **How to create a service**

Example

```
@NgModule({  
  providers: [UserService]  
})
```

OR

```
@Component({  
  providers: [UserService]  
})
```

🎯 Key Point

- Module → shared instance
- Component → new instance

🔑 58. What is InjectionToken?

Used when:

👉 Injecting **non-class dependencies**

Example

```
import { InjectionToken } from '@angular/core';  
  
export const API_URL = new InjectionToken<string>('apiUrl');
```

Provide value

```
providers: [  
  { provide: API_URL, useValue: 'https://api.example.com' }  
]
```

Inject

```
constructor(@Inject(API_URL) private apiUrl: string) {}
```

🎯 Interview Tip

Used for constants or config values

📁 59. What is Singleton Service?

A **singleton service** means:

👉 Only **one instance exists** across the application

Example

```
@Injectable({  
  providedIn: 'root'  
})  
export class AuthService {}
```

Why important?

- Shared data
- Consistent state

🎯 Interview Tip

Default behavior when using providedIn: 'root'

⚙️ 60. How Angular Resolves Dependencies?

☑️ Step-by-step process

1. Component requests a dependency
`constructor(private userService: UserService) {}`
2. Angular looks in **component injector**
3. If not found → checks **parent injectors**
4. Finally → checks **root injector**
5. If found → instance returned
If not → error

Flow

Component → Parent → Root Injector

🎯 Interview Tip

Angular searches dependency in hierarchical order

🚀 Final Summary

- Service → business logic
- DI → inject dependencies
- Injector → provides services
- Hierarchical DI → multiple levels
- providedIn: root → singleton
- InjectionToken → non-class dependency

💧 Scenario 1

👉 "How to create multiple instances of a service?"

☑ Interview Answer (Structured)

By default, Angular creates a **single (singleton) instance** of a service.

To create **multiple instances**, you provide the service at the **component level**.

🤖 Why?

Each component gets its **own injector**, so:

Each component instance → gets its own service instance

☑ Example

Service

```
@Injectable()
export class CounterService {
  count = 0;

  increment() {
    this.count++;
  }
}
```

Component

```
@Component({
  selector: 'app-counter',
  template: `
    <p>{{ counterService.count }}</p>
    <button (click)="counterService.increment()">+</button>
  `,
})
```

```
providers: [CounterService] // 🔥 Important
}))
export class CounterComponent {

  constructor(public counterService: CounterService) {}

}
```

Behavior

Component A → CounterService (instance 1)

Component B → CounterService (instance 2)

👉 Each component has **separate state**

When to Use?

- Independent component state
- Reusable widgets (like counters, forms)

Interview Trap

If you use:

providedIn: 'root'

👉 Then only **one instance** will be shared

Scenario 2

👉 "How to share data globally?"

Interview Answer

Use:

Singleton service (providedIn: 'root')

This ensures:

- 👉 One shared instance across the application

🤔 Why?

All components use the **same service instance**, so data stays consistent.

☑ Example

Service

```
@Injectable({
  providedIn: 'root'
})
export class AuthService {

  userName = "";

  setUser(name: string) {
    this.userName = name;
  }

  getUser() {
    return this.userName;
  }
}
```

Component A (Set Data)

```
constructor(private auth: AuthService) {}

login() {
  this.auth.setUser('Vignesh');
}
```

Component B (Get Data)

```
constructor(private auth: AuthService) {}
```

```
ngOnInit() {  
  console.log(this.auth.getUser());  
}
```

Flow

Component A → Service (shared) → Component B

Interview Tip

providedIn: 'root' creates a singleton service shared across the app

Advanced (Real-world)

Use **BehaviorSubject** for reactive data sharing:

```
private user = new BehaviorSubject<string>('Guest');  
user$ = this.user.asObservable();
```

Scenario 3

"Why use Dependency Injection (DI)?"

Interview Answer (Strong Version)

Dependency Injection is used to:

Decouple components from service creation, making the application more maintainable, testable, and scalable.

Key Benefits

1. Loose Coupling

Component does not create service → reduces dependency

2. Easy Testing

You can replace real service with mock

3. Reusability

Same service used across components

4. Maintainability

Centralized logic

Without DI (Bad Practice)

```
export class UserComponent {  
  service = new UserService(); // tightly coupled   
}
```

With DI (Good Practice)

```
constructor(private service: UserService) {}
```

Testing Example

```
class MockUserService {  
  getUsers() {  
    return ['Test User'];  
  }  
}
```

 Replace real service easily

Interview Tip

DI improves testability and reduces tight coupling

Final Summary (Say This Confidently)

- Multiple instances → provide service at component level
- Global sharing → singleton (providedIn: 'root')
- DI → loose coupling + testability + scalability

 1.  "What happens if service is provided in both root and component?"

Interview Answer

If a service is provided in both:

- providedIn: 'root'
- AND inside a component providers array

 Then:

The component gets its own new instance, overriding the root instance.

Why?

Angular uses **hierarchical injectors**:

Root Injector



Component Injector

When Angular resolves dependency:

1. Checks component injector first
2. If found → uses that instance
3. Ignores root instance

☑ Example

Service

```
@Injectable({  
  providedIn: 'root'  
})  
export class CounterService {  
  count = 0;  
}
```

Component A (Overrides)

```
@Component({  
  selector: 'app-a',  
  template: `  
    <p>A Count: {{ service.count }}</p>  
    <button (click)="service.count++">Increment A</button>  
  `,  
  providers: [CounterService] // 🔥 New instance  
})  
export class ComponentA {  
  constructor(public service: CounterService) {}  
}
```

Component B (Uses root)

```
@Component({
  selector: 'app-b',
  template: `
    <p>B Count: {{ service.count }}</p>
    <button (click)="service.count++">Increment B</button>
  `,
})
export class ComponentB {
  constructor(public service: CounterService) {}
}
```

Behavior

Component A → CounterService (new instance)

Component B → CounterService (root instance)

👉 Changes in A will NOT affect B

Interview Tip

Component-level provider overrides root-level singleton

2. "How to restrict service to a module?"

Interview Answer

To restrict a service to a specific module:

Provide it inside the module's providers array

🤔 Why?

That service will be:

- Available only within that module
- Not shared globally (unless module is imported multiple times)

☑ Example

Service

```
@Injectable()
export class ProductService {
  getProducts() {
    return ['P1', 'P2'];
  }
}
```

Feature Module

```
@NgModule({
  declarations: [],
  providers: [ProductService] // 🔥 Scoped to this module
})
export class ProductModule {}
```

Usage

```
constructor(private productService: ProductService) {}
```

⚠ Important (Interview Trap)

If module is **lazy loaded**:

👉 Each lazy module gets its **own instance**

Best Practice

- Use providedIn: 'root' → global
- Use module providers → scoped logic

3. "Can DI create circular dependency?"

Interview Answer

Yes, **circular dependency occurs when two services depend on each other**, and Angular **cannot resolve it**, resulting in an error.

What is Circular Dependency?

Service A → depends on Service B

Service B → depends on Service A

 Infinite loop 

Example (Problem)

```
@Injectable()
export class ServiceA {
  constructor(private serviceB: ServiceB) {}
}
@Injectable()
export class ServiceB {
  constructor(private serviceA: ServiceA) {}
}
```

Error

Circular dependency detected

☑ Solution 1: Use Injector

Break dependency by resolving manually

```
import { Injector } from '@angular/core';

@Injectable()
export class ServiceA {

  private serviceB: ServiceB;

  constructor(private injector: Injector) {}

  getServiceB() {
    this.serviceB = this.injector.get(ServiceB);
  }
}
```

☑ Solution 2: Use Mediator Service

Create a third service

ServiceA → SharedService ← ServiceB

☑ Solution 3: Refactor Logic

- Move shared logic to another service
- Avoid tight coupling

🎯 Interview Tip

Angular DI cannot resolve circular dependencies automatically

Final Summary (Say This Confidently)

- Component provider overrides root → new instance
- Module providers → scoped service

- Circular dependency → Angular throws error
- Fix using Injector or refactoring

💧 1. 📖 "How to detect circular dependency?"

☑ Interview Answer

Angular detects circular dependency **at runtime** when resolving dependencies via the injector.

👉 If two services depend on each other directly/indirectly:

Angular throws a runtime error like:
Circular dependency detected

✗ Example (Circular Dependency)

```
@Injectable()
export class ServiceA {
  constructor(private serviceB: ServiceB) {}
}
@Injectable()
export class ServiceB {
  constructor(private serviceA: ServiceA) {}
}
```

📖 What happens internally?

Injector tries:

ServiceA → needs ServiceB

ServiceB → needs ServiceA

→ infinite loop → ERROR ✗

☑ How to Detect (Practically)

- App crashes at runtime
- Error in console:

NullInjectorError / Circular dependency detected

Interview Tip

Angular does not allow circular dependency because it cannot resolve infinite loops.

Follow-up Question

 "Can circular dependency happen indirectly?"

 YES

ServiceA → ServiceB → ServiceC → ServiceA

 2.  "Is circular dependency always bad?"

Interview Answer

Yes—circular dependency is considered **bad design** because it creates:

- Tight coupling
- Hard-to-maintain code
- Testing complexity
- Runtime failures

 Why it's bad?

Tight Coupling

ServiceA depends on ServiceB

ServiceB depends on ServiceA

 Neither can work independently

✗ Hard to Test

You cannot mock one without affecting the other

✗ Violates SOLID Principle

- Breaks **Single Responsibility Principle**
- Breaks **Dependency Inversion Principle**

☑ Better Design

Use:

- Shared service
- Interface abstraction
- Event-based communication

☑ Example (Fix using Mediator)

```
@Injectable({ providedIn: 'root' })
export class SharedService {
  data = "";
}
constructor(private shared: SharedService) {}
```

🎯 Interview Tip

Circular dependency indicates poor design and should be refactored.

Follow-up Question

👉 "Is there any case where circular dependency is acceptable?"

✓ Rarely—but:

- Can be handled using Injector
- But still not recommended

💧 3. 👉 "How Angular resolves dependencies?"

Interview Answer

Angular resolves dependencies using a **hierarchical injector system**.

Resolution Flow

When a component requests a service:

```
constructor(private service: MyService) {}
```

Angular checks in order:

1. Component Injector
2. Parent Component Injector
3. Module Injector
4. Root Injector

Example

Service

```
@Injectable({  
  providedIn: 'root'  
})  
export class LoggerService {  
  log(msg: string) {
```

```
    console.log(msg);  
  }  
}
```

Component

```
constructor(private logger: LoggerService) {}  
  
ngOnInit() {  
  this.logger.log('Hello');  
}
```

Internal Flow

Component asks → Injector checks:

- Component providers? ❌
- Module providers? ❌
- Root injector? ✅
- Returns instance

Key Concept

Angular always searches **top-down (child → parent → root)**

Follow-up Questions

 "What happens if service not found anywhere?"

✔ Angular throws:

NullInjectorError: No provider for Service

👉 "How to control which instance is used?"

✓ Provide service at:

- Component level → new instance
- Root → shared instance

👉 "What is lazy-loaded module behavior?"

✓ Each lazy module has its **own injector**

👉 So service can have **multiple instances**

👉 "How would you break circular dependency in Angular?"

☑ Interview Answer (Strong)

There are multiple ways:

☑ Solution 1: Use Injector (Lazy Injection)

```
import { Injector } from '@angular/core';

@Injectable()
export class ServiceA {

  private serviceB!: ServiceB;

  constructor(private injector: Injector) {}

  getServiceB() {
    if (!this.serviceB) {
      this.serviceB = this.injector.get(ServiceB);
    }
    return this.serviceB;
  }
}
```

👉 Delays dependency resolution

☑ **Solution 2: Use Mediator (Best Practice)**

ServiceA → SharedService ← ServiceB

☑ **Solution 3: Use RxJS (Event-driven)**

```
private subject = new BehaviorSubject<any>(null);
```

☑ **Solution 4: Refactor Responsibilities**

👉 Move shared logic into another service

🎯 **Best Answer to Say**

The best way is to refactor design and remove tight coupling instead of forcing DI to work.

🚀 **Final Summary (Interview Ready)**

- Circular dependency → runtime error
- Always bad → indicates poor design
- Angular resolves dependencies → hierarchical injectors
- Fix circular dependency → refactor or use mediator

61. What is HttpClient?

☑ **Interview Answer**

HttpClient is an Angular service used to:

Communicate with backend APIs (GET, POST, PUT, DELETE)

It is part of `@angular/common/http` and returns **Observables**.

☑ Example

```
import { HttpClient } from '@angular/common/http';

constructor(private http: HttpClient) {}

getUsers() {
  return this.http.get('/api/users');
}
```

🎯 Key Points

- Returns **Observable**
- Supports interceptors
- Handles JSON automatically

🏰 62. Difference between HttpClient and Http

Feature	Http (Old)	HttpClient (New)
Module	@angular/http	@angular/common/http
Status	Deprecated ❌	Active ✅
JSON parsing	Manual	Automatic
Interceptors	❌	✅
Typing	Weak	Strong

🎯 Interview Tip

Always use HttpClient (Http is deprecated)

63. How to make GET request?

Example

```
getUsers() {  
  return this.http.get<any[]>('/api/users');  
}
```

Usage

```
ngOnInit() {  
  this.getUsers().subscribe(data => {  
    console.log(data);  
  });  
}
```

Best Practice

Use interface:

```
interface User {  
  id: number;  
  name: string;  
}
```

64. How to handle POST request?

Example

```
addUser(user: any) {  
  return this.http.post('/api/users', user);  
}
```

Usage

```
this.addUser({ name: 'Vignesh' })  
  .subscribe(res => console.log(res));
```

Interview Tip

POST sends data to server (create operation)

65. What is Interceptor?

Interview Answer

An interceptor is a class that:

Intercepts HTTP requests and responses globally

Use Cases

- Add JWT token
- Logging
- Error handling

Example

```
import { HttpInterceptor } from '@angular/common/http';
```

```
@Injectable()
```

```
export class AuthInterceptor implements HttpInterceptor {
```

```
  intercept(req: HttpRequest<any>, next: HttpHandler) {
```

```
    const cloned = req.clone({  
      headers: req.headers.set('Authorization', 'Bearer token')  
    });
```

```
    return next.handle(cloned);  
  }  
}
```

Interview Tip

Interceptors act like middleware

66. How to add headers to requests?

Method 1: Direct

```
this.http.get('/api/users', {  
  headers: { Authorization: 'Bearer token' }  
});
```

Method 2: Using HttpHeaders

```
import { HttpHeaders } from '@angular/common/http';
```

```
const headers = new HttpHeaders()  
  .set('Authorization', 'Bearer token');
```

```
this.http.get('/api/users', { headers });
```

Best Practice

Use interceptor instead of manually adding headers everywhere

⚠ 67. How to handle errors globally?

☑ Interview Answer

Use:

HTTP Interceptor + catchError

☑ Example

```
import { catchError } from 'rxjs/operators';
import { throwError } from 'rxjs';

intercept(req: HttpRequest<any>, next: HttpHandler) {
  return next.handle(req).pipe(
    catchError(error => {
      console.error('Error:', error);
      return throwError(() => error);
    })
  );
}
```

🎯 Benefits

- Centralized error handling
- Clean components

📄 68. What is Retry Logic?

☑ Interview Answer

Retry logic automatically:

Retries failed HTTP requests

☑ Example

```
import { retry } from 'rxjs/operators';
```

```
this.http.get('/api/users')  
  .pipe(retry(3))  
  .subscribe();
```

🎯 Use Case

- Temporary network issues

⚠ Warning

Avoid retry for:

- Authentication errors (401)
- Bad requests (400)

📖 69. What is async pipe?

☑ Interview Answer

async pipe automatically:

Subscribes and unsubscribes from Observables in template

☑ Example

```
users$ = this.http.get('/api/users');  
<ul>  
  <li *ngFor="let user of users$ | async">  
    {{ user.name }}  
  </li>  
</ul>
```

🎯 Benefits

- No manual subscribe
- No memory leaks

🏆 70. Difference between subscribe vs async pipe

Feature	subscribe()	async pipe
Location	Component	Template
Unsubscribe	Manual	Automatic
Use case	Logic handling	UI binding
Memory leak risk	High ❌	Low ✅

✅ Example

subscribe()

```
this.http.get('/api/users')  
  .subscribe(data => this.users = data);
```

async pipe

```
<li *ngFor="let user of users$ | async">
```

🎯 Interview Tip

Prefer async pipe for UI, subscribe for logic

Real Interview Scenarios

 "How to attach token to every API call?"

✓ Use interceptor

 "How to avoid memory leaks in API calls?"

✓ Use async pipe or unsubscribe

 "How to handle API failure?"

✓ catchError + interceptor

Final Summary

- HttpClient → API communication
- Interceptor → global request/response handling
- Headers → attach auth data
- Retry → reattempt failed calls
- async pipe → auto subscribe/unsubscribe

71. What is Observable?

Interview Answer

An **Observable** is a stream of data that:

emits values over time asynchronously

Key Points

- Lazy (runs only when subscribed)

- Can emit multiple values
- Supports operators (map, filter, etc.)

Example

```
import { Observable } from 'rxjs';

const obs$ = new Observable(observer => {
  observer.next(1);
  observer.next(2);
  observer.next(3);
  observer.complete();
});

obs$.subscribe(value => console.log(value));
```

Output

```
1
2
3
```

Interview Tip

Observable = multiple values over time

72. Difference between Observable and Promise

Feature	Observable	Promise
Values	Multiple	Single
Lazy	Yes	No
Cancel	Yes	No
Operators	Yes	No

Execution On Immediately
subscribe

☑ Example

Promise

```
const p = new Promise(resolve => resolve(1));
```

👉 Executes immediately

Observable

```
const obs$ = new Observable(obs => obs.next(1));
```

👉 Executes only on subscribe

🎯 Interview Tip

Observable is more powerful than Promise

🔗 73. What is Subscription?

☑ Interview Answer

A **subscription** is:

The execution of an Observable

☑ Example

```
const sub = obs$.subscribe(value => console.log(value));
```

Key Point

Observable + subscribe() → execution starts

74. Why unsubscribe?

Interview Answer

To:

Prevent memory leaks and unwanted background execution

Problem

```
this.http.get('/api').subscribe();
```

 Keeps listening even after component destroyed

Solution

subscription: any;

```
ngOnInit() {  
  this.subscription = this.http.get('/api').subscribe();  
}
```

```
ngOnDestroy() {  
  this.subscription.unsubscribe();  
}
```

Interview Tip

Always unsubscribe for long-lived observables

75. What is Subject?

Interview Answer

A **Subject** is:

Both an Observable AND Observer

Meaning

- Can emit values
- Can subscribe to values

Example

```
import { Subject } from 'rxjs';

const subject = new Subject<number>();

subject.subscribe(val => console.log('A:', val));
subject.subscribe(val => console.log('B:', val));

subject.next(1);
subject.next(2);
```

Output

```
A: 1
B: 1
A: 2
B: 2
```

Interview Tip

Subject = multicast (multiple subscribers)

76. Types of Subjects

1. Subject
2. BehaviorSubject
3. ReplaySubject
4. AsyncSubject

77. What is BehaviorSubject?

Interview Answer

BehaviorSubject:

Stores the **latest value** and emits it to new subscribers

Example

```
import { BehaviorSubject } from 'rxjs';

const subject = new BehaviorSubject<number>(0);

subject.subscribe(val => console.log('A:', val));

subject.next(1);

subject.subscribe(val => console.log('B:', val));
```

Output

A: 0
A: 1
B: 1

Key Point

New subscribers get last value immediately

78. What is ReplaySubject?

Interview Answer

ReplaySubject:

Replays previous values to new subscribers

Example

```
import { ReplaySubject } from 'rxjs';

const subject = new ReplaySubject<number>(2);

subject.next(1);
subject.next(2);
subject.next(3);

subject.subscribe(val => console.log(val));
```

Output

2
3

Key Point

Buffer size controls how many values are replayed

79. What is AsyncSubject?

Interview Answer

AsyncSubject:

Emits **only the last value** when completed

Example

```
import { AsyncSubject } from 'rxjs';

const subject = new AsyncSubject<number>();

subject.subscribe(val => console.log(val));

subject.next(1);
subject.next(2);
subject.next(3);

subject.complete();
```

Output

3

Key Point

Emits last value ONLY after complete()

80. What is pipe?

Interview Answer

pipe() is used to:

Apply operators to Observables

Example

```
import { map, filter } from 'rxjs/operators';
```

```
this.http.get<number[]>('/api').pipe(  
  filter(x => x > 10),  
  map(x => x * 2)  
)  
.subscribe(console.log);
```

Key Point

Observable → pipe → operators → result

Real Interview Scenarios

"How to share data between components?"

✓ BehaviorSubject

"Why use Subject instead of Observable?"

✓ To manually emit values

"Which Subject to use?"

Case	Use
Shared state	BehaviorSubject
Replay values	ReplaySubject

Last value
only AsyncSubject

Final Summary

- Observable → async data stream
- Subscription → execution
- Subject → multicast
- BehaviorSubject → latest value
- ReplaySubject → previous values
- AsyncSubject → last value only
- pipe → apply operators

81. What is map?

Interview Answer

map transforms each emitted value into a new value.

Example

```
import { of } from 'rxjs';  
import { map } from 'rxjs/operators';  
  
of(1, 2, 3).pipe(  
  map(x => x * 10)  
).subscribe(console.log);
```

Output

```
10  
20  
30
```

🎯 Use Case

- Transform API response
- Modify data

💧 82. What is filter?

☑ Interview Answer

filter emits only values that match a condition.

☑ Example

```
import { filter } from 'rxjs/operators';
```

```
of(1, 2, 3, 4).pipe(  
  filter(x => x % 2 === 0)  
).subscribe(console.log);
```

🎯 Output

2
4

🎯 Use Case

- Ignore unwanted values
- Validate data

💧 83. What is tap?

☑ Interview Answer

tap is used for **side effects** without modifying data.

☑ Example

```
import { tap } from 'rxjs/operators';

of(1, 2).pipe(
  tap(x => console.log('Before:', x))
).subscribe(console.log);
```

🎯 Use Case

- Logging
- Debugging
- Triggering actions

🎯 Key Point

tap does NOT change data

💧 84. What is switchMap?

☑ Interview Answer

switchMap:

Cancels previous request and switches to new one

☑ Example (Search API)

```
searchInput.valueChanges.pipe(
  switchMap(term => this.http.get(`/api/search?q=${term}`))
).subscribe(console.log);
```

🎯 Use Case

- Search autocomplete
- Live API calls

🎯 Key Point

Only latest request matters

💧 85. What is mergeMap?

☑ Interview Answer

mergeMap:

Executes multiple inner observables **in parallel**

☑ Example

```
of(1, 2, 3).pipe(  
  mergeMap(id => this.http.get(`/api/user/${id}`))  
).subscribe(console.log);
```

🎯 Use Case

- Multiple API calls
- Parallel processing

💧 86. Difference: switchMap vs mergeMap

Feature	switchMap	mergeMap
Cancels previous	✅ Yes	❌ No

Parallel calls	 No	 Yes
Use case	Search	Batch API
Performance	Optimized	Heavy

Example

switchMap

A → canceled

B → result

mergeMap

A → result

B → result

Interview Tip

switchMap = latest only

mergeMap = all requests

87. What is concatMap?

Interview Answer

concatMap:

Executes requests **one after another (sequentially)**

Example

```
of(1, 2, 3).pipe(
  concatMap(id => this.http.get(`/api/user/${id}`))
).subscribe(console.log);
```

Use Case

- Order-sensitive operations
- Payment processing

Key Point

1 → done → 2 → done → 3

88. What is debounceTime?

Interview Answer

debounceTime:

Delays execution until user stops typing

Example

```
searchInput.valueChanges.pipe(  
  debounceTime(500)  
).subscribe(console.log);
```

Use Case

- Search input
- Reduce API calls

Key Point

Waits before emitting value

89. What is distinctUntilChanged?

Interview Answer

Prevents duplicate consecutive values.

Example

```
of(1, 1, 2, 2, 3).pipe(  
  distinctUntilChanged()  
).subscribe(console.log);
```

Output

```
1  
2  
3
```

Use Case

- Avoid repeated API calls

90. What is catchError?

Interview Answer

Handles errors in observable stream.

Example

```
import { catchError } from 'rxjs/operators';  
import { of } from 'rxjs';  
  
this.http.get('/api').pipe(  
  catchError((err) => of('Error: ' + err.message))  
).subscribe(console.log);
```

```
catchError(err => {  
  console.error(err);  
  return of([]); // fallback value  
})  
).subscribe(console.log);
```

Use Case

- API error handling
- Fallback response

Real Interview Scenarios (VERY IMPORTANT)

"User typing search input"

✓ Answer:

debounceTime + distinctUntilChanged + switchMap

"Call multiple APIs in parallel"

✓ Answer:

mergeMap

"Call APIs in order"

✓ Answer:

concatMap

"Cancel previous API request"

✓ Answer:

switchMap

Final Summary (Say This Confidently)

- map → transform
- filter → condition
- tap → side effects
- switchMap → cancel previous
- mergeMap → parallel
- concatMap → sequential
- debounceTime → delay
- distinctUntilChanged → avoid duplicates
- catchError → handle errors

91. What is Angular Routing?

Interview Answer

Angular routing is a mechanism that:

Enables navigation between different views/components without reloading the page (SPA)

Example

/login → LoginComponent

/dashboard → DashboardComponent

Key Point

Angular updates the view dynamically instead of reloading the page

⚙️ 92. How to configure routes?

☑️ Step 1: Define routes

```
import { Routes } from '@angular/router';

const routes: Routes = [
  { path: 'login', component: LoginComponent },
  { path: 'dashboard', component: DashboardComponent },
  { path: '', redirectTo: 'login', pathMatch: 'full' }
];
```

Step 2: Import RouterModule

```
import { RouterModule } from '@angular/router';

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

🎯 Interview Tip

Routes map URL → Component

⚙️ 93. What is router-outlet?

☑️ Interview Answer

router-outlet is a directive that:

Acts as a placeholder where routed components are displayed

Example

```
<router-outlet></router-outlet>
```

Flow

URL → Router → Component → router-outlet renders it

94. What is routerLink?

Interview Answer

routerLink is used to:

Navigate between routes declaratively in templates

Example

```
<a routerLink="/dashboard">Go to Dashboard</a>
```

Dynamic Example

```
<a [routerLink]="['/user', userId]">User</a>
```

Interview Tip

Prefer routerLink over href

95. What is Lazy Loading?

Interview Answer

Lazy loading means:

Loading modules only when needed instead of loading everything at startup

Why?

- Improves performance
- Reduces initial load time

Example

```
const routes: Routes = [  
  {  
    path: 'admin',  
    loadChildren: () =>  
      import('./admin/admin.module').then(m => m.AdminModule)  
  }  
];
```

Interview Tip

Lazy loading is used for large applications

96. What are Route Guards?

Interview Answer

Route guards are used to:

Control access to routes

Example

- Only logged-in users can access dashboard

97. Types of Route Guards

1. **CanActivate** → allow/block route
2. **CanActivateChild** → child routes
3. **CanDeactivate** → prevent leaving
4. **Resolve** → fetch data before route
5. **CanLoad** → prevent lazy loading

Interview Tip

Most used: CanActivate

98. What is AuthGuard?

Interview Answer

AuthGuard is a guard that:

Allows access only if user is authenticated

Example

```
import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';
```

```
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
```

```
  constructor(private router: Router) {}
```

```
  canActivate(): boolean {
    const isLoggedIn = !!localStorage.getItem('token');
```

```
    if (!isLoggedIn) {
      this.router.navigate(['/login']);
```

```
    return false;
  }

  return true;
}
```

Route Usage

```
{ path: 'dashboard', component: DashboardComponent, canActivate: [AuthGuard] }
```

Interview Tip

AuthGuard is used for route protection

99. What is ActivatedRoute?

Interview Answer

ActivatedRoute provides:

Information about the current route (params, query params, data)

Example

URL

/user/10

Component

```
import { ActivatedRoute } from '@angular/router';

constructor(private route: ActivatedRoute) {}
```

```
ngOnInit() {
  const id = this.route.snapshot.params['id'];
  console.log(id);
}
```

Use Cases

- Route parameters
- Query parameters

100. Difference between snapshot and observable

Feature	snapshot	observable
Type	Static	Dynamic
Updates	 No	 Yes
Use case	One-time read	Continuous updates

Example

Snapshot

```
const id = this.route.snapshot.params['id'];
```

Observable

```
this.route.params.subscribe(params => {
  console.log(params['id']);
});
```

Key Difference

snapshot → one-time value

observable → listens to changes

Real Interview Scenarios

"Protect routes from unauthorized users"

- ✓ Use AuthGuard

"Load large modules efficiently"

- ✓ Use Lazy Loading

"Get dynamic route param"

- ✓ Use ActivatedRoute

"User navigates same route with different id"

- ✓ Use observable (not snapshot)

Final Summary

- Routing → navigation without reload
- router-outlet → renders component
- routerLink → navigation
- Lazy loading → performance
- Guards → security
- ActivatedRoute → route data

101. Template-driven vs Reactive Forms

Template-driven Forms

- Defined in **HTML**
- Uses ngModel
- Simple forms

Example

```
<input [(ngModel)]="name" required>
```

Reactive Forms

- Defined in **TypeScript**
- Uses FormControl, FormGroup
- More scalable

Example

```
this.form = new FormGroup({  
  name: new FormControl("")  
});
```

Difference

Feature	Template	Reactive
Control	HTML	TS
Validation	Limited	Powerful
Scalability	Low	High

Interview Tip

Use Reactive forms for real-world apps

◇ 102. What is FormControl?

☒ Interview Answer

Represents a **single input field**

Example

```
name = new FormControl("");
```

HTML

```
<input [formControl]="name">
```

◇ 103. What is FormGroup?

☒ Interview Answer

A collection of multiple FormControls

Example

```
this.form = new FormGroup({  
  name: new FormControl(""),  
  email: new FormControl("")  
});
```

HTML

```
<form [formGroup]="form">  
  <input formControlName="name">
```

```
<input formControlName="email">
</form>
```

◇ 104. What is FormArray?

☒ Interview Answer

Used for **dynamic list of controls**

Example

```
this.form = new FormGroup({
  skills: new FormArray([
    new FormControl("")
  ])
});
```

Add dynamically

```
(this.form.get('skills') as FormArray).push(new FormControl(""));
```

◇ 105. What are Validators?

☒ Interview Answer

Validators are functions used to:

Validate form inputs

Built-in Validators

```
import { Validators } from '@angular/forms';

name: new FormControl("", Validators.required)
```

Common validators

- required
- minlength
- maxlength
- email

◇ 106. How to create custom validator?

☒ Example

```
import { AbstractControl } from '@angular/forms';

export function noSpaceValidator(control: AbstractControl) {
  if (control.value && control.value.includes(' ')) {
    return { noSpace: true };
  }
  return null;
}
```

Usage

```
name: new FormControl("", [noSpaceValidator])
```

◊ 107. What is FormBuilder?

☑ Interview Answer

FormBuilder is a helper service to:

Simplify form creation

Example

```
import { FormBuilder } from '@angular/forms';

constructor(private fb: FormBuilder) {}

this.form = this.fb.group({
  name: [''],
  email: ['']
});
```

🎯 Benefit

Less boilerplate code

◊ 108. What is valueChanges?

☑ Interview Answer

An observable that:

Emits value whenever form changes

Example

```
this.form.valueChanges.subscribe(value => {  
  console.log(value);  
});
```

Use Case

- Real-time validation
- API calls on input change

◇ 109. patchValue vs setValue

setValue

- Must provide all fields

```
this.form.setValue({  
  name: 'Vignesh',  
  email: 'test@mail.com'  
});
```

patchValue

- Partial update allowed

```
this.form.patchValue({  
  name: 'Vignesh'  
});
```

Difference

Feature	setValue	patchValue
All fields	Required	Optional
Flexibility	Low	High

Interview Tip

patchValue is more commonly used

◇ 110. How to handle dynamic forms?

Interview Answer

Use:

FormArray

Example

TS

```
this.form = this.fb.group({  
  skills: this.fb.array([])  
});
```

Add field

```
addSkill() {  
  (this.form.get('skills') as FormArray).push(this.fb.control(""));  
}
```

HTML

```
<div formArrayName="skills">  
  <div *ngFor="let skill of form.get('skills').controls; let i = index">  
    <input [formControlName]="i">  
  </div>  
</div>  
  
<button (click)="addSkill()">Add Skill</button>
```

Use Case

- Dynamic forms
- Add/remove fields

Real Interview Scenarios

"Form with dynamic fields?"

✓ FormArray

"Real-time validation?"

✓ valueChanges

"Partial update?"

✓ patchValue

"Large complex form?"

✓ Reactive Forms + FormBuilder

Final Summary

- FormControl → single field
- FormGroup → group of fields
- FormArray → dynamic fields

- Validators → validation
- FormBuilder → simplify forms
- valueChanges → listen to changes
- patchValue → partial update

111. How Angular handles authentication?

Interview Answer

Angular itself doesn't handle authentication directly. Instead, it:

Uses backend APIs (e.g., .NET) + JWT + services + guards + interceptors

Flow

Login → API → JWT Token → Store Token → Attach to requests → Validate on server

Example

```
login(credentials: any) {  
  return this.http.post('/api/login', credentials);  
}
```

After login:

```
localStorage.setItem('token', response.token);
```

Interview Tip

Angular handles auth using JWT + HttpInterceptor + Route Guards

112. What is JWT?

Interview Answer

JWT (JSON Web Token) is:

A secure token used to **authenticate users between client and server**

Structure

Header.Payload.Signature

Example

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Key Point

- Stateless authentication
- Sent in headers

113. Where to store token?

Options

Storage	Secure?	Notes
localStorage	✗	vulnerable to XSS
sessionStorage	✗	slightly better
HttpOnly Cookie	✓	safest

Best Practice

Use **HttpOnly cookies** (cannot be accessed by JS)

114. How to secure routes?

Interview Answer

Use:

Route Guards (AuthGuard)

Example

```
canActivate(): boolean {  
  const token = localStorage.getItem('token');  
  
  if (!token) {  
    this.router.navigate(['/login']);  
    return false;  
  }  
  
  return true;  
}
```

Route

```
{ path: 'dashboard', canActivate: [AuthGuard] }
```

Tip

Guards prevent unauthorized access

115. What is interceptor in auth?

Interview Answer

Auth interceptor:

Automatically attaches JWT token to every request

Example

```
intercept(req: HttpRequest<any>, next: HttpHandler) {  
  
  const token = localStorage.getItem('token');  
  
  const cloned = req.clone({  
    setHeaders: {  
      Authorization: `Bearer ${token}`  
    }  
  });  
  
  return next.handle(cloned);  
}
```

Benefit

- No need to manually attach token everywhere

116. What is XSS?

Interview Answer

XSS (Cross-Site Scripting):

A vulnerability where attacker injects malicious JavaScript

✖ Example

```
<div>{{ userInput }}</div>
```

If userInput = `<script>alert('hack')</script>`

🎯 Impact

- Steal tokens
- Execute scripts

⚠ 117. What is CSRF?

☑ Interview Answer

CSRF (Cross-Site Request Forgery):

Attacker tricks user into making unwanted requests

Example

User logged in → attacker triggers request → server executes it

🎯 Protection

- CSRF tokens
- SameSite cookies

🛡 118. How Angular prevents XSS?

☑ Interview Answer

Angular uses:

Built-in sanitization

Example

```
<div [innerHTML]="data"></div>
```

👉 Angular removes unsafe scripts automatically

🎯 Key Point

Angular escapes HTML by default

🔗 119. What is DomSanitizer?

☑ Interview Answer

Used to:

Bypass Angular security for trusted content

☑ Example

```
import { DomSanitizer } from '@angular/platform-browser';

constructor(private sanitizer: DomSanitizer) {}

safeHtml = this.sanitizer.bypassSecurityTrustHtml('<b>Hello</b>');
```

⚠ Warning

Use carefully (can expose XSS risk)

120. How to handle refresh tokens?

Interview Answer

Refresh tokens are used to:

Get a new access token when it expires

Flow

Access Token expires → Send Refresh Token → Get new Access Token

121. What is Change Detection Strategy?

Interview Answer

Change detection is how Angular:

Updates the DOM when data changes

Types

1. Default

- Checks entire component tree
- Runs frequently

2. OnPush

- Checks only when:
 - Input changes
 - Event occurs
 - Observable emits

☑ Example

```
import { ChangeDetectionStrategy } from '@angular/core';

@Component({
  selector: 'app-user',
  template: `{{ user.name }}`,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class UserComponent {
  @Input() user: any;
}
```

🎯 Interview Tip

OnPush improves performance by reducing unnecessary checks

⚙ 122. What is zone.js?

☑ Interview Answer

zone.js is a library that:

Tracks async operations and triggers change detection

🔄 Example

setTimeout → zone.js detects → Angular updates UI

🎯 Key Point

Angular knows when to update UI because of zone.js

123. What is AOT vs JIT?

☒ AOT (Ahead of Time)

- Compiled at build time
- Faster performance

☒ JIT (Just in Time)

- Compiled in browser
- Used in development

Difference

Feature	AOT	JIT
Compile	Build time	Runtime
Speed	Fast	Slower
Bundle size	Small	Larger

Interview Tip

Production uses AOT

124. What is Tree Shaking?

☒ Interview Answer

Tree shaking:

Removes unused code during build

Example

```
import { map } from 'rxjs/operators';
```

 Only used operators included

Benefit

- Smaller bundle
- Faster load

125. What is Lazy Loading Modules?

(Already covered but here's advanced view)

Interview Answer

Lazy loading:

Loads modules only when route is accessed

Example

```
{  
  path: 'admin',  
  loadChildren: () =>  
    import('./admin/admin.module').then(m => m.AdminModule)  
}
```

Benefit

- Improves initial load time

126. What is trackBy?

Interview Answer

trackBy helps Angular:

Identify items in list efficiently

Without trackBy

- Re-renders entire list

Example

```
<li *ngFor="let user of users; trackBy: trackById">
  {{ user.name }}
</li>

trackById(index: number, user: any) {
  return user.id;
}
```

Benefit

- Better performance in large lists

127. What is Pure vs Impure Pipe?

Pure Pipe

- Runs only when input changes

Impure Pipe

- Runs on every change detection

Example

```
@Pipe({  
  name: 'myPipe',  
  pure: false  
})
```

Interview Tip

Avoid impure pipes (performance issue)

128. What is Memoization?

Interview Answer

Memoization:

Caching function results to avoid recalculation

Example

```
const cache: any = {};
```

```
function sum(n: number) {  
  if (cache[n]) return cache[n];  
  
  let result = n * 2;  
  cache[n] = result;  
  return result;  
}
```

Use Case

- Expensive calculations
- Pipes optimization

129. How to optimize Angular app?

☒ Interview Answer (Strong)

Use:

Key Techniques

1. OnPush Change Detection

changeDetection: ChangeDetectionStrategy.OnPush

2. Lazy Loading

- Load modules on demand

3. trackBy in ngFor

- Avoid re-rendering

4. Use async pipe

- Avoid manual subscriptions

5. Avoid unnecessary API calls

- Use caching

6. Use AOT build

ng build --prod

7. Minimize bundle size

- Tree shaking
- Remove unused libraries

Interview Tip

Combine multiple techniques for performance

130. What is Web Worker?

Interview Answer

Web Worker:

Runs heavy tasks in **background thread**

Why?

- Prevent UI blocking

Example

ng generate web-worker app

Worker Code

```
addEventListener('message', ({ data }) => {  
  const result = data * 2;  
  postMessage(result);  
});
```

Component

```
worker.postMessage(10);
```

```
worker.onmessage = ({ data }) => {  
  console.log(data);  
};
```

Use Case

- Heavy calculations
- Image processing

Real Interview Scenarios

 "App is slow, how to improve?"

✓ OnPush + Lazy Loading + trackBy

 "Large list rendering issue?"

✓ trackBy

 "Heavy computation blocking UI?"

✓ Web Worker

 "Why Angular updates UI automatically?"

✓ zone.js

Final Summary

- Change Detection → updates DOM
- OnPush → performance boost
- zone.js → tracks async
- AOT → faster build
- Tree shaking → smaller bundle
- trackBy → efficient lists
- Memoization → caching
- Web Worker → background tasks

131. What is Unit Testing?

Interview Answer

Unit testing is:

Testing individual parts (units) of code in isolation

Example

Testing a function:

```
function add(a: number, b: number) {  
  return a + b;  
}
```

Test:

```
it('should add two numbers', () => {  
  expect(add(2, 3)).toBe(5);  
});
```

Key Point

Test smallest piece of logic independently

132. What is Jasmine?

Interview Answer

Jasmine is a:

Testing framework used to write test cases in Angular

Features

- describe() → test suite
- it() → test case
- expect() → assertion

Example

```
describe('Math Test', () => {  
  it('should add numbers', () => {  
    expect(2 + 2).toBe(4);  
  });  
});
```

Interview Tip

Jasmine is like xUnit/NUnit in .NET

⚙️ 133. What is Karma?

☑️ Interview Answer

Karma is a:

Test runner that executes tests in browser

🔄 Flow

Write test → Karma runs → Browser executes → Result shown

🎯 Interview Tip

Karma runs Jasmine tests

🔗 134. How to test a component?

☑️ Interview Answer

Use:

TestBed + ComponentFixture

☑️ Example

Component

```
@Component({
  selector: 'app-test',
  template: `<h1>{{ title }}</h1>`
})
export class TestComponent {
```

```
title = 'Hello';  
}
```

Test

```
import { TestBed } from '@angular/core/testing';  
  
describe('TestComponent', () => {  
  let fixture: any;  
  let component: TestComponent;  
  
  beforeEach(() => {  
    TestBed.configureTestingModule({  
      declarations: [TestComponent]  
    });  
  
    fixture = TestBed.createComponent(TestComponent);  
    component = fixture.componentInstance;  
    fixture.detectChanges();  
  });  
  
  it('should display title', () => {  
    const compiled = fixture.nativeElement;  
    expect(compiled.querySelector('h1').textContent).toContain('Hello');  
  });  
});
```

Interview Tip

Always call detectChanges() to update DOM

135. What is TestBed?

Interview Answer

TestBed is:

Angular's testing utility to configure testing environment

What it does

- Creates module for testing
- Injects dependencies
- Creates components

Example

```
TestBed.configureTestingModule({  
  declarations: [MyComponent],  
  providers: [MyService]  
});
```

Interview Tip

Think of TestBed as a mini Angular module for testing

136. What is Mocking?

Interview Answer

Mocking means:

Replacing real dependencies with fake ones for testing

Why?

- Avoid real API calls
- Control behavior

Example

```
class MockService {  
  getData() {  
    return ['Test Data'];  
  }  
}
```

Use in Test

```
providers: [{ provide: DataService, useClass: MockService }]
```

Interview Tip

Mocking improves test reliability

137. How to test HTTP calls?

Interview Answer

Use:

HttpClientTestingModule + HttpTestingController

Example

```
import { HttpClientTestingModule, HttpTestingController } from  
'@angular/common/http/testing';  
  
describe('UserService', () => {  
  let service: any;  
  let httpMock: HttpTestingController;  
  
  beforeEach(() => {  
    TestBed.configureTestingModule({
```

```
imports: [HttpClientTestingModule],
providers: [UserService]
});

service = TestBed.inject(UserService);
httpMock = TestBed.inject(HttpTestingController);
});

it('should fetch users', () => {
  service.getUsers().subscribe(data => {
    expect(data.length).toBe(1);
  });

  const req = httpMock.expectOne('/api/users');
  req.flush([ { name: 'Vignesh' } ]);
});
});
```

Flow

Call service → intercept request → mock response → assert result

Interview Tip

Never call real APIs in unit tests

Real Interview Scenarios

"How to test component with service?"

✓ Mock the service

👉 "How to test API calls?"

- ✓ HttpTestingController

👉 "Why use mocking?"

- ✓ Avoid external dependency

👉 "How Angular testing works?"

- ✓ TestBed + Jasmine + Karma

🚀 Final Summary

- Unit Testing → test small units
- Jasmine → test framework
- Karma → test runner
- TestBed → testing module
- Mocking → fake dependencies
- HTTP testing → HttpTestingController

☑ Example (Interceptor)

```
intercept(req: HttpRequest<any>, next: HttpHandler) {  
  return next.handle(req).pipe(  
    catchError(error => {  
      if (error.status === 401) {  
        return this.refreshToken().pipe(  
          switchMap(newToken => {  
            const cloned = req.clone({  
              setHeaders: { Authorization: `Bearer ${newToken}` }  
            });  
            return next.handle(cloned);  
          });  
        }  
      }  
    });  
  }  
}
```

```
        })
    );
}
return throwError(() => error);
})
);
}
```

Best Practice

- Store refresh token securely (HttpOnly cookie)
- Keep access token short-lived

Real Interview Scenarios

"How to attach token to every API?"

✓ Interceptor

"How to protect routes?"

✓ AuthGuard

"Where to store token securely?"

✓ HttpOnly cookie

"How to handle token expiry?"

✓ Refresh token flow

Final Summary (Interview Ready)

- Angular auth → JWT + Guards + Interceptors
- JWT → stateless token
- Secure routes → AuthGuard
- Token attach → Interceptor
- XSS → script injection
- CSRF → fake request
- DomSanitizer → bypass security
- Refresh token → renew access

138. How to avoid multiple API calls while typing?

☒ Interview Answer

Use:

debounceTime + distinctUntilChanged + switchMap

☒ Example

```
searchControl.valueChanges.pipe(  
  debounceTime(500),  
  distinctUntilChanged(),  
  switchMap(term => this.http.get(`/api/search?q=${term}`))  
).subscribe(console.log);
```

Why?

- debounceTime → wait for user to stop typing
- distinctUntilChanged → ignore duplicate values
- switchMap → cancel previous request

139. How to share data across components?

Answer

Use:

Service + BehaviorSubject

Example

```
private data = new BehaviorSubject<string>('default');  
data$ = this.data.asObservable();
```

```
update(value: string) {  
  this.data.next(value);  
}
```

Why?

- Real-time updates
- Shared state

140. How to cache API responses?

Answer

Use:

shareReplay() operator

Example

```
getUsers() {  
  if (!this.users$) {  
    this.users$ = this.http.get('/api/users').pipe(  
      shareReplay(1)
```

```
        shareReplay(1)
    );
}
return this.users$;
}
```

🎯 Why?

- Avoid duplicate API calls
- Improve performance

💧 141. How to handle global error handling?

☑ Answer

Use:

HTTP Interceptor + catchError

☑ Example

```
intercept(req: HttpRequest<any>, next: HttpHandler) {
    return next.handle(req).pipe(
        catchError(error => {
            console.error(error);
            return throwError(() => error);
        })
    );
}
```

🎯 Why?

- Centralized error handling
- Cleaner components

142. How to implement role-based access?

Answer

Use:

AuthGuard + Roles

Example

```
canActivate(): boolean {  
  const role = localStorage.getItem('role');  
  
  if (role !== 'admin') {  
    this.router.navigate(['/unauthorized']);  
    return false;  
  }  
  
  return true;  
}
```

Route

```
{ path: 'admin', canActivate: [AuthGuard] }
```

Advanced Tip

Store roles in JWT

143. How to optimize large list rendering?

Answer

Use:

trackBy + virtual scrolling

Example

```
<li *ngFor="let item of items; trackBy: trackById">  
trackById(index: number, item: any) {  
  return item.id;  
}
```

Advanced

Use Angular CDK Virtual Scroll

144. How to cancel previous API request?

Answer

Use:

switchMap

Example

```
searchControl.valueChanges.pipe(  
  switchMap(term => this.http.get(`/api?q=${term}`))  
)  
).subscribe();
```

Why?

Automatically cancels previous request

💧 145. How to handle file upload?

☑ Answer

Use:

FormData + HttpClient

☑ Example

```
upload(file: File) {  
  const formData = new FormData();  
  formData.append('file', file);  
  
  return this.http.post('/api/upload', formData);  
}
```

HTML

```
<input type="file" (change)="onFileSelected($event)">
```

🎯 Tip

Use progress tracking:

```
{ reportProgress: true, observe: 'events' }
```

💧 146. How to implement pagination?

☑ Answer

Use:

page + size parameters

☑ Example

```
getUsers(page: number) {  
  return this.http.get(`/api/users?page=${page}&size=10`);  
}
```

🌀 UI Flow

Page 1 → Page 2 → Page 3

💧 147. How to implement infinite scroll?

☑ Answer

Use:

scroll event + API calls

☑ Example

```
onScroll() {  
  this.page++;  
  this.loadData();  
}
```

Advanced (Better)

Use RxJS:

```
fromEvent(window, 'scroll').pipe(  
  debounceTime(200)  
).subscribe(() => this.loadMore());
```

Use Case

- Social feeds
- Large datasets

Real Interview Follow-ups

"Difference: Pagination vs Infinite Scroll?"

Pagination	Infinite Scroll
Page-based	Continuous
Better control	Better UX

"How to avoid duplicate API calls?"

- ✓ shareReplay / caching

"How to improve search performance?"

- ✓ debounceTime + switchMap

Final Summary (Say This Confidently)

- Avoid API spam → debounceTime + switchMap
- Share data → BehaviorSubject
- Cache → shareReplay
- Errors → interceptor
- Role access → AuthGuard
- Large list → trackBy

- Cancel request → switchMap
- File upload → FormData
- Pagination → page params
- Infinite scroll → scroll + API

💧 1. Angular Pipes (VERY COMMON)

? What is a Pipe?

☑ Answer

A Pipe is used to:

Transform data in templates

☑ Example

```
{{ price | currency }}
{{ date | date:'short' }}
```

🎯 Built-in Pipes

- date
- currency
- uppercase
- json

💧 Custom Pipe

```
import { Pipe, PipeTransform } from '@angular/core';
```

```
@Pipe({ name: 'double' })
export class DoublePipe implements PipeTransform {
  transform(value: number): number {
    return value * 2;
  }
}
```

```
}  
}
```

Usage

```
{{ 5 | double }} <!-- Output: 10 -->
```

🎯 Interview Tip

Pipes are used only in templates, not in TS logic

💧 2. Angular Guards Deep Scenario (Advanced)

? Difference: CanLoad vs CanActivate

Guard	Purpose
CanActivate	Prevent route access
CanLoad	Prevent module loading

🎯 Example

```
canLoad(): boolean {  
  return false; // module won't load  
}
```

🎯 Interview Trick Question

CanActivate still downloads module ❌

CanLoad prevents download ✅

3. Angular Resolvers (VERY IMPORTANT)

What is Resolver?

Answer

Resolver:

Fetches data BEFORE route loads

Example

```
resolve() {  
  return this.http.get('/api/users');  
}
```

Route

```
{  
  path: 'users',  
  component: UserComponent,  
  resolve: { users: UserResolver }  
}
```

Component

```
this.route.data.subscribe(data => {  
  console.log(data['users']);  
});
```

Why Important?

Avoid loading spinner in UI

💧 4. Angular Signals (NEW & TRENDING 💧)

? What are Signals?

☑ Answer

Signals are:

Reactive state management introduced in Angular

☑ Example

```
import { signal } from '@angular/core';
```

```
count = signal(0);
```

```
increment() {  
  this.count.set(this.count() + 1);  
}
```

HTML

```
{{ count() }}
```

🎯 Interview Tip

Signals reduce dependency on RxJS for state

💧 5. Standalone Components (IMPORTANT)

? What are Standalone Components?

☑ Answer

Components that:

Do not require NgModule

Example

```
@Component({
  standalone: true,
  selector: 'app-test',
  template: `<h1>Hello</h1>`
})
export class TestComponent {}
```

Why Important?

- Cleaner architecture
- Modern Angular approach

6. Angular Lifecycle Deep Question

Order of Lifecycle Hooks

Answer

```
constructor
ngOnChanges
ngOnInit
ngDoCheck
ngAfterContentInit
ngAfterContentChecked
ngAfterViewInit
ngAfterViewChecked
ngOnDestroy
```

Interview Trick

ngOnChanges runs BEFORE ngOnInit

7. Change Detection Deep Scenario

? Why UI not updating in OnPush?

☒ Answer

Because:

Angular checks only reference changes

Wrong

```
this.user.name = 'New'; // won't update
```

☒ Correct

```
this.user = { ...this.user, name: 'New' };
```

Interview Tip

Immutable updates are required

8. RxJS Subject vs BehaviorSubject (Frequently Asked)

Feature	Subject	BehaviorSubject
Initial value		
Last value		

Use case Events State

🎯 Interview Answer

Use BehaviorSubject for shared state

💧 9. Angular Module Types

? Types of Modules

- Root Module (AppModule)
- Feature Module
- Shared Module
- Core Module

🎯 Example

```
@NgModule({  
  declarations: [],  
  imports: [],  
  exports: []  
})  
export class SharedModule {}
```

🎯 Interview Tip

SharedModule = reusable components

CoreModule = singleton services

💧 10. Environment Configuration

? What is environment.ts?

☑ Answer

Used for:

Managing different configs (dev, prod)

Example

```
export const environment = {  
  apiUrl: 'http://localhost:5000'  
};
```

🎯 Interview Tip

Used for API URLs and keys

💧 11. Angular Folder Structure (Real Project)

☑ Ideal Structure

```
src/  
├── app/  
│   ├── core/      (services, guards)  
│   ├── shared/    (reusable components)  
│   ├── features/  (modules)  
│   ├── models/  
│   └── services/
```

Interview Tip

Structure shows your experience level

12. State Management (NgRx – Advanced)

What is NgRx?

Answer

NgRx is:

Redux-based state management for Angular

When to use?

- Large apps
- Complex state

13. SSR (Server Side Rendering)

What is Angular Universal?

Answer

Renders Angular app on server

Benefits

- SEO
- Faster load

14. Web Storage vs Cookies

Feature	localStorage	Cookies
Size	Large	Small
Security	Low	High (HttpOnly)